# 汇编语言第三次作业

## 课堂练习

**课堂练习题目：进制转换**

基本要求：输入 1 到 2 位 16 进制数，输出对应的 10 进制数。
追加要求：实现 10 十进制向 16 进制转换。

结果显示：




这个题目是汇编语言里的进制转换题目分成两个部分：

- 基本要求： 输入 1～2 位十六进制数，例如 `ff` ，输出对应的十进制数 `255` 。
- 追加要求： 实现十进制转十六进制，例如输入 `255` ，输出 `FF` 。

## 一、十六进制转十进制

思路很简单：先把输入的十六进制字符转换成数值，再计算十进制结果。

## 二、编写汇编代码

### 第 1 步：新建文件

在 `D:\myassembly` 文件夹中新建文件，命名为 `exchange.asm` 。

写代码：

代码块

```
1
2   .model small
3   .stack 100h
4
5   .data
6       prompt db 'Please input a Hex number: $'
7       result db 0Dh,0Ah,'The Dec number:',0Dh,0Ah,'$'
8
```

```
.code
START:
    mov ax, @data
    mov ds, ax

    mov ah, 09h
    lea dx, prompt
    int 21h

    xor bx, bx
    mov cx, 2

READ_HEX:
    mov ah, 01h
    int 21h

    cmp al, 0Dh
    je CONVERT_DONE

    cmp al, '0'
    jb INVALID
    cmp al, '9'
    jbe NUMERIC

    and al, 0DFh
    cmp al, 'A'
    jb INVALID
    cmp al, 'F'
    ja INVALID
    sub al, 'A'-10
    jmp ADD_DIGIT

NUMERIC:
    sub al, '0'

ADD_DIGIT:
    shl bx, 1
    shl bx, 1
    shl bx, 1
    shl bx, 1
    xor ah, ah
    add bx, ax
    loop READ_HEX

CONVERT_DONE:
    mov ah, 09h
    lea dx, result
```

```
    int 21h

    mov ax, bx
    mov bx, 10
    xor cx, cx

    cmp ax, 0
    jne DIV_LOOP

    mov dl, '0'
    mov ah, 02h
    int 21h
    jmp EXIT

DIV_LOOP:
    xor dx, dx
    div bx
    push dx
    inc cx
    cmp ax, 0
    jne DIV_LOOP

PRINT_LOOP:
    pop dx
    add dl, '0'
    mov ah, 02h
    int 21h
    loop PRINT_LOOP

    jmp EXIT

INVALID:
    mov dx, offset result
    mov ah, 09h
    int 21h

EXIT:
    mov ax, 4C00h
    int 21h

END START

```

## 三、编译并运行

打开你之前使用的 DOSBox，依次输入下面的命令。

第 1 步：汇编。

代码块

```
masm exchange.asm
```

```
D:\>masm exchange.asm
Microsoft (R) Macro Assembler Version 5.00
Copyright (C) Microsoft Corp 1981-1985, 1987.  All rights reserved.

Object filename [exchange.OBJ]:
Source listing  [NUL.LST]:
Cross-reference [NUL.CRF]:

  51534 + 465010 Bytes symbol space free

      0 Warning Errors
      0 Severe  Errors
```

第 2 步：链接。

代码块

```
link exchange.obj
```

```
D:\>link exchange.obj

Microsoft (R) Overlay Linker  Version 3.60
Copyright (C) Microsoft Corp 1983-1987.  All rights reserved.

Run File [EXCHANGE.EXE]:
List File [NUL.MAP]:
Libraries [.LIB]:
```

第 3 步：运行。

代码块

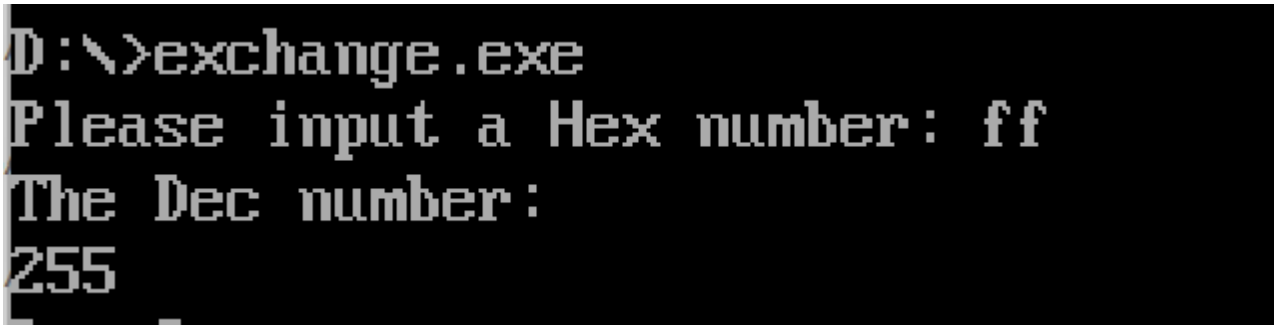
```
exchange.exe
```

## 四、测试结果

例如输入 `ff`，预期显示：

代码块

```
Please input a Hex number:
ff
The Dec number:
255
```




## 作业三

# 作业题目 3：求和

基本要求：求 1+2+……+100，并将结果“5050”打印到屏幕。




注意和的结果数据表示范围，结果的进制转换等问题。

1）›尝试结果分别放在寄存器中、放在数据段中，放在栈中等不同位置的操作；

2）›用户输入 1~100 内的任何一个数，完成十进制结果输出。（查找 21 号中断的功能表，找到输入数据的功能调用）

3）›用 C 语言实现后察看反汇编代码并加注释；

一共要完成四项内容：

1. 用汇编计算 $1+2+\cdots+100$，输出 `5050`。
2. 尝试把求和结果分别放在寄存器、数据段和堆栈中。
3. 让用户输入 `1～100` 之间的任意整数，计算并输出十进制结果。
4. 用 C 语言实现，并查看编译生成的汇编代码、添加注释。

## 第一部分：计算 1 到 100 的和

思路很简单：用一个寄存器保存当前数字，另一个寄存器累计求和。

- 当前数字从 1 开始。
- 每次把当前数字加到总和中。
- 当前数字加 1。
- 重复 100 次。

- 最后将结果 `5050` 转换为十进制字符并输出。

注意，计算结果 5050 超过了 8 位寄存器的范围，所以我们使用 16 位寄存器 `AX` 保存结果。

## 第 1 步：创建 `sum.asm`

在 `D:\myassembly` 文件夹中新建文件 `sum.asm`

写代码：

代码块

```
.model small
.stack 100h

.code
START:
    mov ax, @data
    mov ds, ax

    mov si, 0
    mov bx, 1
    mov cx, 100

SUM_LOOP:
    add si, bx
    inc bx
    loop SUM_LOOP

    mov ax, si
    mov bx, 10
    xor cx, cx

CONVERT_LOOP:
    xor dx, dx
    div bx
    push dx
    inc cx
    cmp ax, 0
    jne CONVERT_LOOP

PRINT_LOOP:
    pop dx
    add dl, '0'
    mov ah, 02h
    int 21h
    loop PRINT_LOOP

    mov ax, 4C00h
```

```
38      int 21h
39
40  END START
```

这里的核心是：

| 寄存器 | 作用 |
| --- | --- |
| SI | 累加求和结果 |
| BX | 当前要累加的数字 |
| CX | 控制循环 100 次 |
| AX | 保存最终结果，并参与十进制转换 |
| DX | 保存每次除以 10 得到的余数 |

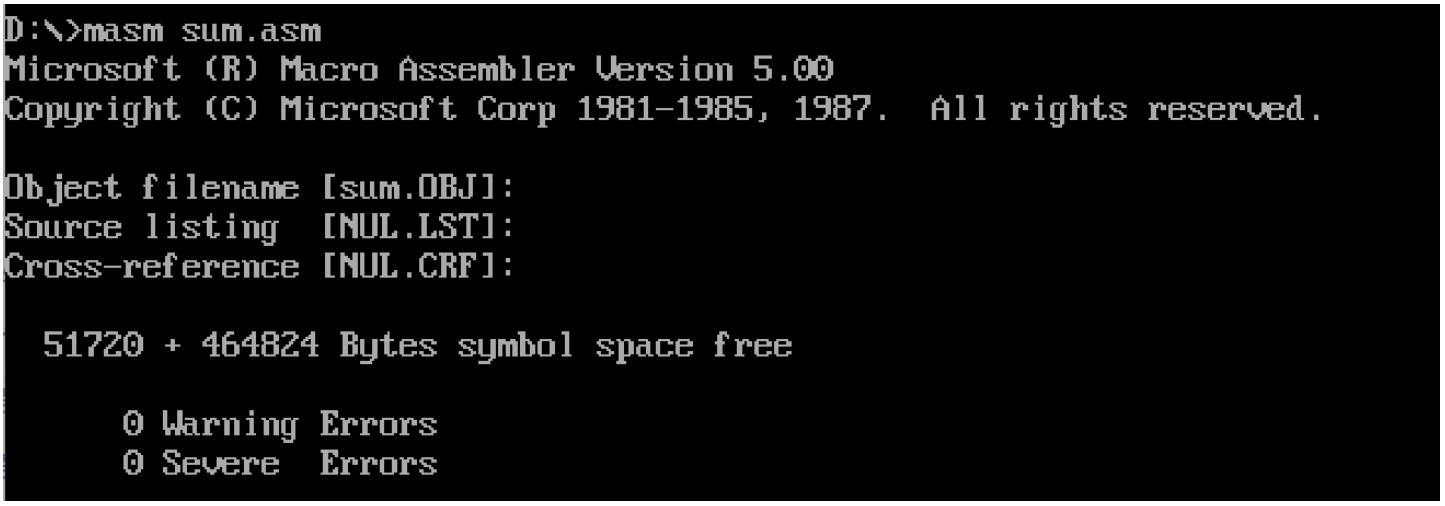
十进制转换时，`5050` 不会直接变成字符串。程序反复除以 10，得到余数 `0、5、0、5`，再通过堆栈倒序输出，最终得到 `5050`。

## 第 2 步：编译和运行

打开 DOSBox，依次输入：

代码块

```
masm sum.asm
```




出现文件名提示时按回车接受默认值。然后输入：

代码块

```
link sum.obj
```

```
D:\>link sum.obj

Microsoft (R) Overlay Linker  Version 3.60
Copyright (C) Microsoft Corp 1983-1987.  All rights reserved.

Run File [SUM.EXE]:
List File [NUL.MAP]:
Libraries [.LIB]:
```

同样按回车接受默认选项。最后运行：

```
代码块
1   sum.exe
```

预期结果：




## 第二部分：尝试把结果存放在不同位置

### 方式一：结果存放在寄存器中

刚刚完成的程序就是这种方式。

```
代码块
1   mov si, 0
2   mov bx, 1
3   mov cx, 100
4
5   SUM_LOOP:
6       add si, bx
7       inc bx
8       loop SUM_LOOP
```

循环结束后，`SI = 5050`，结果保存在寄存器中。

### 方式二：结果存放在数据段中

新建文件 `sumdata.asm`，写代码：

代码块

```
.model small
.stack 100h

.data
    SUM DW 0

.code
START:
    mov ax, @data
    mov ds, ax

    mov si, 0
    mov bx, 1
    mov cx, 100

SUM_LOOP:
    add si, bx
    inc bx
    loop SUM_LOOP

    mov SUM, si

    mov ax, SUM
    mov bx, 10
    xor cx, cx

CONVERT_LOOP:
    xor dx, dx
    div bx
    push dx
    inc cx
    cmp ax, 0
    jne CONVERT_LOOP

PRINT_LOOP:
    pop dx
    add dl, '0'
    mov ah, 02h
    int 21h
    loop PRINT_LOOP

    mov ax, 4C00h
    int 21h

END START
```

这里最关键的是：

```
代码块
1   mov SUM, si
```

它把寄存器 `SI` 中的结果写入数据段变量 `SUM`。之后使用 `mov ax, SUM` 读取结果，再输出。

编译运行：

```
代码块
1   masm sumdata.asm
2   link sumdata.obj
3   sumdata.exe
```

预期输出仍然是 `5050`。




## 方式三：结果存放在堆栈中

新建文件 `sumstack.asm`，写代码：

```
代码块
1   .model small
2   .stack 100h
3
4   .code
5   START:
6       mov ax, @data
7       mov ds, ax
8
9       mov si, 0
10      mov bx, 1
11      mov cx, 100
12
13  SUM_LOOP:
14      add si, bx
15      inc bx
16      loop SUM_LOOP
17
18      push si
```

```
    pop ax

    mov bx, 10
    xor cx, cx

CONVERT_LOOP:
    xor dx, dx
    div bx
    push dx
    inc cx
    cmp ax, 0
    jne CONVERT_LOOP

PRINT_LOOP:
    pop dx
    add dl, '0'
    mov ah, 02h
    int 21h
    loop PRINT_LOOP

    mov ax, 4C00h
    int 21h

END START
```

关键语句：

代码块

```
push si
pop ax
```

这两条指令把求和结果压入堆栈，再从堆栈取出到 `AX`。需要注意，后续十进制转换也使用堆栈保存余数，因此这里先取出求和结果，再开始转换。

编译运行：

代码块

```
masm sumstack.asm
link sumstack.obj
sumstack.exe
```

预期输出仍然是 `5050`。




## 三种方式的区别

| 方式 | 保存结果的位置 | 核心指令 |
| --- | --- | --- |
| 寄存器 | SI 等寄存器 | mov、add |
| 数据段 | 内存变量 SUM | mov SUM, si |
| 堆栈 | 堆栈内存 | push、pop |

## 第三部分：用户输入 1～100 的整数并求和

## 第一步：新建 `suminput.asm`

在 `D:\myassembly` 文件夹中新建文件 `suminput.asm`。

代码块

```
.model small
.stack 100h

.data
    prompt  db 'Please input a number (1-100): $'
    result  db 0Dh,0Ah,'The sum is: $'
    errorMsg db 0Dh,0Ah,'Invalid input!$'
    newline db 0Dh,0Ah,'$'

.code
START:
    mov ax, @data
    mov ds, ax

    mov ah, 09h
    lea dx, prompt
    int 21h

    xor di, di

READ_INPUT:
    mov ah, 01h
    int 21h

    cmp al, 0Dh
```

```
26      je INPUT_DONE
27
28      cmp al, '0'
29      jb INVALID
30      cmp al, '9'
31      ja INVALID
32
33      sub al, '0'
34      xor ah, ah
35      push ax
36
37      mov ax, di
38      mov bx, 10
39      mul bx
40      mov di, ax
41
42      pop ax
43      add di, ax
44      cmp di, 100
45      ja INVALID
46
47      jmp READ_INPUT
48
49  INPUT_DONE:
50      cmp di, 1
51      jb INVALID
52
53      xor si, si
54      mov bx, 1
55
56  SUM_LOOP:
57      add si, bx
58      inc bx
59      cmp bx, di
60      jbe SUM_LOOP
61
62      mov ah, 09h
63      lea dx, result
64      int 21h
65
66      mov ax, si
67      mov bx, 10
68      xor cx, cx
69
70  CONVERT_LOOP:
71      xor dx, dx
72      div bx
```

```
    push dx
    inc cx
    cmp ax, 0
    jne CONVERT_LOOP

PRINT_LOOP:
    pop dx
    add dl, '0'
    mov ah, 02h
    int 21h
    loop PRINT_LOOP

    mov ah, 09h
    lea dx, newline
    int 21h

    mov ax, 4C00h
    int 21h

INVALID:
    mov ah, 09h
    lea dx, errorMsg
    int 21h

    mov ah, 09h
    lea dx, newline
    int 21h

    mov ax, 4C01h
    int 21h

END START
```

## 第二步：编译并运行

打开 DOSBox，依次输入下面三条命令：

代码块

```
masm suminput.asm
```

代码块

```
link suminput.obj
```

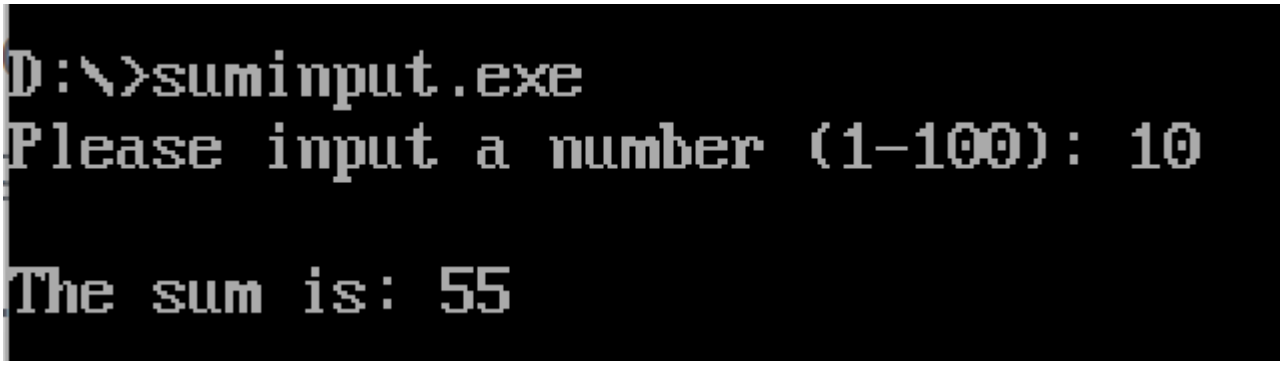
代码块

```
1   suminput.exe
```

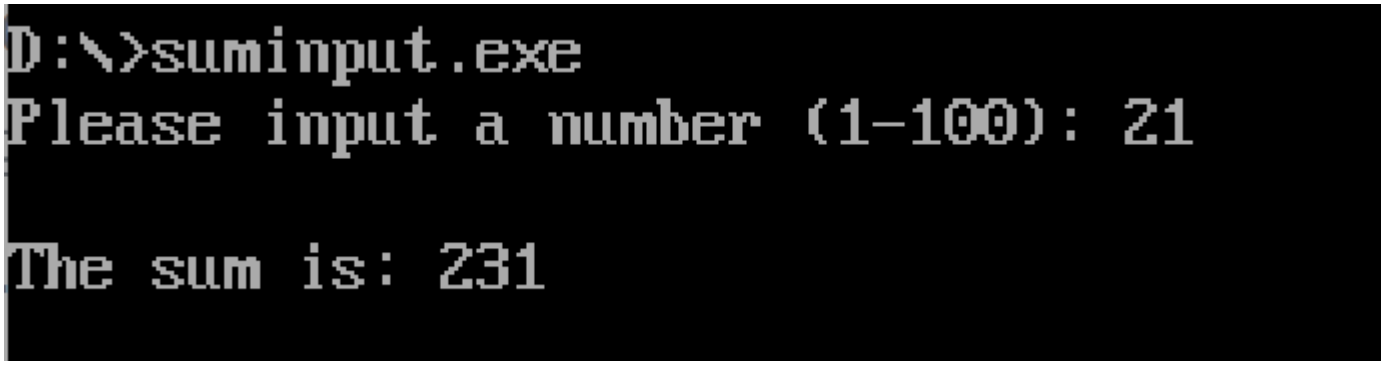
## 第三步：测试程序

运行后会出现输入提示，你可以测试以下几组数据。

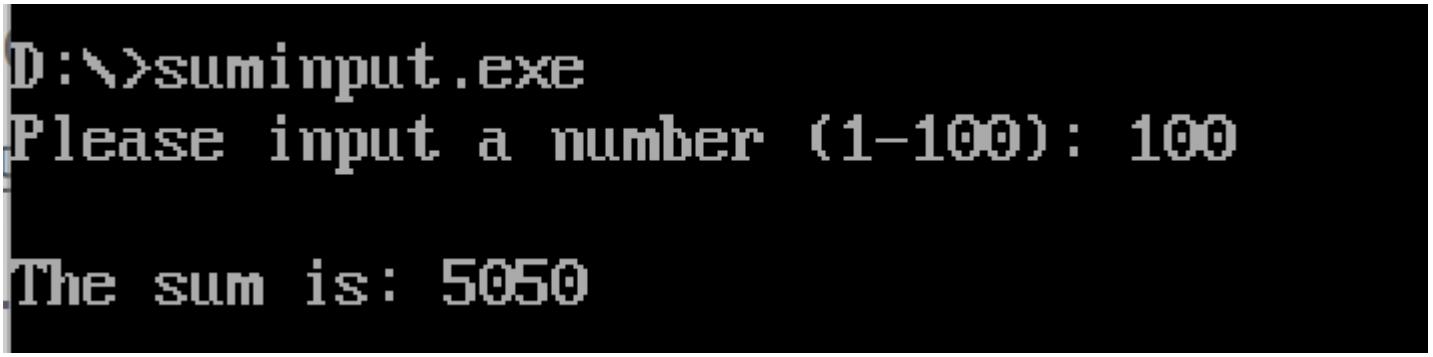
测试一：输入10，预期结果为55



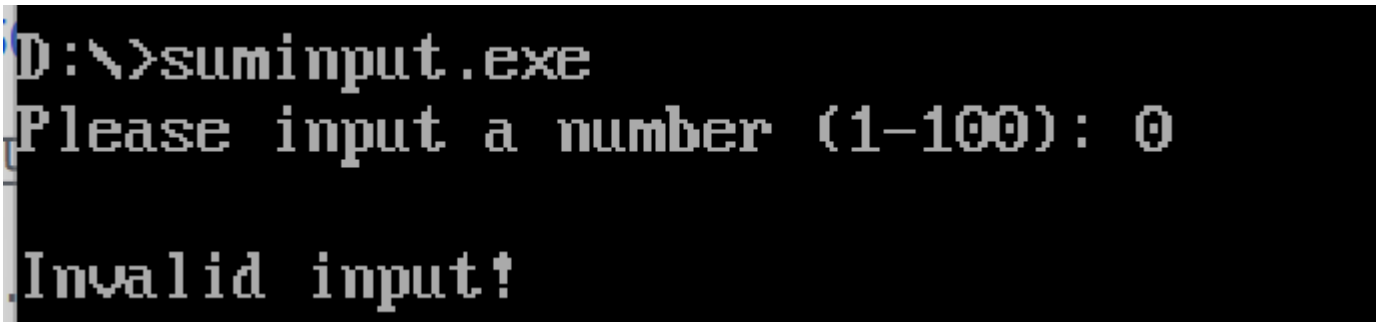

测试二：输入21，预期结果为231

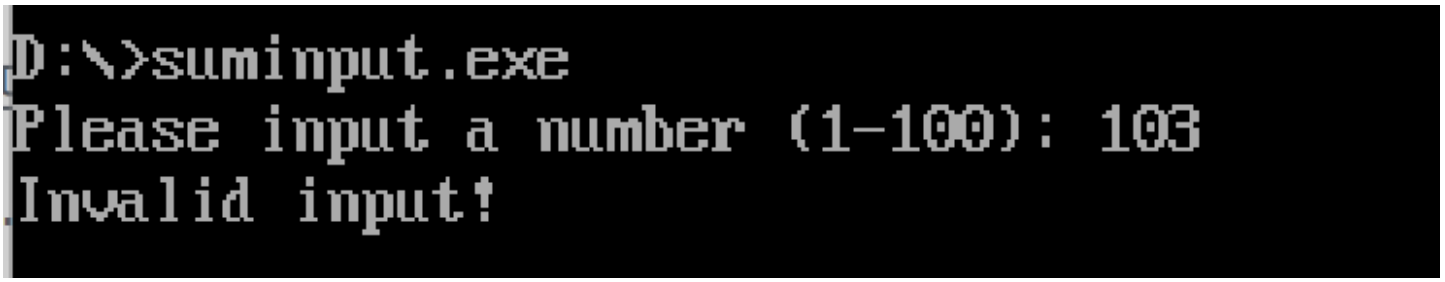



测试三：输入100，预期结果为5050




输入数字后按回车，程序就会计算并输出结果。如果输入 `0`、大于 `100` 的数字或非数字字符，程序会提示 `Invalid input!`。




## 第四步：理解这部分的核心代码

### 1. 为什么可以输入多位数字?

代码块

```
1   mov ax, di
2   mov bx, 10
3   mul bx
4   mov di, ax
5   add di, ax
```

这段代码的实际用途是将之前输入的数乘以 10，再加上当前数字。例如输入 `21`：

- 先输入 `2`，当前数值为 2。
- 再输入 `1`，原数值乘以 10 得到 20，再加 1，得到 21。

### 2. 如何计算总和?

代码块

```
1   xor si, si
2   mov bx, 1
3
4   SUM_LOOP:
5       add si, bx
6       inc bx
7       cmp bx, di
8       jbe SUM_LOOP
```

其中 `DI` 保存用户输入的数字，`SI` 保存累加结果，`BX` 从 1 开始递增，直到加完用户输入的数字。

### 3. 为什么需要十进制转换？

计算结果保存在二进制数值寄存器中，不能直接把它当作屏幕上的数字显示。因此，程序使用除以 10 的方法获取各位数字，再借助堆栈倒序输出。

这份代码的输入逻辑使用 `DI` 保存输入值，计算结果使用 `SI` 保存，因此结果最大为 5050 时不会溢出。

## 第四部分：C 语言实现并查看汇编代码

### 第一步：编写 C 语言程序

在 `D:\myassembly` 文件夹中新建 `sum.c`，写代码：

代码块

```
#include <stdio.h>

int main(void)
{
    int sum = 0;
    int i;

    for (i = 1; i <= 100; i++)
    {
        sum += i;
    }

    printf("%d\n", sum);

    return 0;
}
```

这段程序和第一部分的汇编程序完成相同的任务：把 1 到 100 累加起来并输出结果。

## 第二步：运行 C 程序

打开 Windows 命令提示符（CMD），输入：

代码块

```
cd /d D:\myassembly
gcc sum.c -o sum_c.exe
```

```
3   sum_c.exe
```

正常情况下，屏幕输出：

```
代码块
1   5050
```

```
D:\myassembly>gcc sum.c -o sum_c.exe

D:\myassembly>sum_c.exe
5050
```

## 第三步：生成汇编代码

在刚才的 CMD 窗口中输入：

```
代码块
1   gcc -S -masm=intel -O0 sum.c -o sum_gcc.s
```

参数含义：

- `-S`：只生成汇编代码，不生成可执行文件。
- `-masm=intel`：使用 Intel 汇编语法，更方便你和之前的 MASM 代码对照。
- `-O0`：关闭优化，让循环逻辑更容易观察。
- `-o sum_gcc.s`：指定生成的文件名。

然后打开 `D:\myassembly\sum_gcc.s`，用 VS Code加注释。

```
    .file   "sum.c"                 # 源文件名称
    .intel_syntax noprefix          # 使用 Intel 汇编语法
    .text                           # 开始代码段
    .section .rdata,"dr"            # 定义只读数据段
.LC0:
    .ascii "%d\12\0"                # 定义输出格式字符串：整数加换行符
    .text
    .globl  "main"                  # 声明 main 为全局符号
    .def    "main"; .scl    2;  .type   32; .endef
    .seh_proc   "main"              # 开始定义 main 函数的异常处理信息
"main":
    push    rbp                     # 保存原来的基址指针
    .seh_pushreg    rbp
    mov rbp, rsp                    # 建立当前函数的栈帧
    .seh_setframe   rbp, 0
    sub rsp, 48                     # 在栈上分配 48 字节空间
    .seh_stackalloc 48
    .seh_endprologue
    call    "__main"                # 初始化运行环境
    mov DWORD PTR -4[rbp], 0        # 将局部变量 sum 初始化为 0
    mov DWORD PTR -8[rbp], 1        # 将局部变量 i 初始化为 1
    jmp .L2                         # 跳转到循环条件判断处

.L3:
    mov eax, DWORD PTR -8[rbp]      # 将变量 i 的值读入 EAX 寄存器
    add DWORD PTR -4[rbp], eax      # sum = sum + i，将 i 累加到 sum
    add DWORD PTR -8[rbp], 1        # i = i + 1，循环变量加 1

.L2:
    cmp DWORD PTR -8[rbp], 100      # 比较 i 和 100
    jle .L3                         # 如果 i <= 100，则跳转到 .L3 执行循环体

    mov eax, DWORD PTR -4[rbp]      # 将最终求和结果 sum 读入 EAX
    lea rcx, .LC0[rip]              # 将格式字符串 "%d\n" 的地址放入 RCX
    mov edx, eax                    # 将求和结果作为 printf 的第二个参数
    call    "printf"                # 调用 printf，输出求和结果

    mov eax, 0                      # 将返回值设为 0，表示程序正常结束
    add rsp, 48                     # 释放之前分配的 48 字节栈空间
    pop rbp                         # 恢复原来的基址指针
    ret                             # 返回调用者
    .seh_endproc                    # 结束 main 函数的异常处理信息

    .def    "__main";   .scl    2;  .type   32; .endef
    .ident  "GCC: (MinGW-W64 x86_64-ucrt-posix-seh, built by Brecht Sanders, r1) 16.2.0"
    .def    "printf";   .scl    2;  .type   32; .endef
```